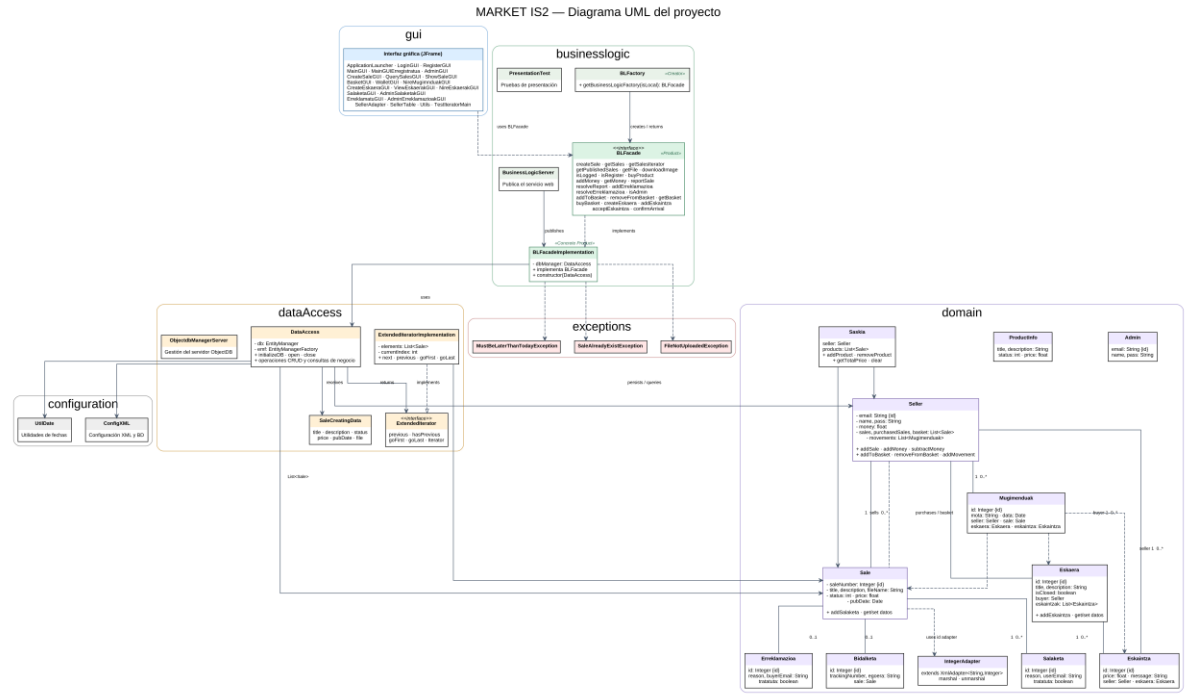
PATTERNS

Github: https://github.com/Oihane870/Market_IS2

Oihane Zubiaurre, Julen Haro y Oiher Bernet

A) Un diagrama UML extendido, indicando los cambios realizados y la finalidad de cada clase/interfaz.



Lo cambios realizados han sido los siguientes:

- Businesslogic
  - Se modificó BLFacade
  - Se creó BLFactory
  - Se modificó BLFacadeImplementation
  - Se modificó PresentationTest
  - Se creó TestIteratorMain
- Configuration
  - No se ha modificado nada
- DataAccess
  - Se creó ExtendedIterator
  - Se creó ExtendedIteratorImplementation
- Domain
  - Se modificó AdapterTest
  - Se modificó Admin

- Se modificó Sale
- Se creó SellerAdapter
- Se creó SellerTable

- Exceptions
  - No se ha modificado nada
- Gui
  - Se modificó MainGui
  - Se creó TestIteratorMain
  - Se modificó ApplicationLauncher

B) El código que habéis modificado, describiendo las líneas más significativas.

- Businesslogic
  - BLFacade
    - Código:

```
    @WebMethod
    public ExtendedIterator getSalesIterator(String desc);
```

  - BLFactory
    - Código:

```
public class BLFactory {

    public BLFactory(){}

    public BLFacade getBusinessLogicFactory(boolean isLocal) {
        ConfigXML c = ConfigXML.getInstance();

        if (isLocal) {
            DataAccess da = new DataAccess();
            return new BLFacadeImplementation(da);
        } else {
            try {
                String serviceName = "http://" + c.getBusinessLogicNode() + ":" + c.getBusinessLogicPort()
                                     + "/ws/" + c.getBusinessLogicName() + "?wsdl";
                URL url = new URL(serviceName);
                QName qname = new QName("http://businessLogic/", "BLFacadeImplementationService");
                Service service = Service.create(url, qname);

                return service.getPort(BLFacade.class);
            } catch (Exception e) {
                e.printStackTrace();
                return null;
            }
        }
    }

}
```

- BLFacadeImplementation
  - Código:

```java
@Override
@WebMethod
public ExtendedIterator getSalesIterator(String desc) {
    dbManager.open();
    // Obtenemos la lista de ventas y la envolvemos correctamente en el iterador
    List<Sale> sales = dbManager.getSales(desc);
    ExtendedIterator iterator = new dataAccess.ExtendedIteratorImplementation(sales);
    dbManager.close();
    return iterator;
}
```

- PresentationTest
  - Código:

```java
package businesslogic;

import org.junit.Test;

public class PresentationTest {

    @Test
    public void testMainGUI() {
        // 1. Usar la implementación real en lugar de un Mock
        BLFacade businessLogic = new BLFacadeImplementation();

        // 2. Inyectar la fachada de negocio en la interfaz gráfica
        // (Asegúrate de que este método estático exista en tu clase MainGUI)
        MainGUI.setBussinessLogic(businessLogic);

        // 3. Instanciar la interfaz gráfica pasándole null (o el contenedor adecuado)
        MainGUI gui = new MainGUI(null);

        // 4. Aserción para verificar que se creó correctamente
        assert gui != null;
    }
}
```

- TestIteratorMain
  - Código:

```java
public class TestIteratorMain {

    public static void main(String[] args) {
        try {
            // 1. Obtenemos la lógica de negocio mediante la factoría
            boolean isLocal = true;
            BLFactory factory = new BLFactory();
            BLFacade blFacade = factory.getBusinessLogicFactory(isLocal);

            // (Omitimos initializeBD() para evitar duplicados en la base de datos)

            // 2. Obtenemos el iterador extendido pasando una cadena vacía ""
            ExtendedIterator it = (ExtendedIterator) blFacade.getSalesIterator("");
            Sale s;

            System.out.println("_____________________");
            System.out.println("FROM LAST TO FIRST");
            it.goLast(); // Ir al último elemento
            while (it.hasPrevious()) {
                s = (Sale) it.previous();
                System.out.println(s);
            }

            System.out.println();
            System.out.println("_____________________");
            System.out.println("FROM FIRST TO LAST");
            it.goFirst(); // Ir al primer elemento
            while (it.hasNext()) {
                s = (Sale) it.next();
                System.out.println(s);
            }

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
```

- DataAccess
  - ExtendedIterator
    - Código:

```
package dataAccess;

import java.util.Iterator;

public interface ExtendedIterator extends Iterator {

    // Return the actual element and go to the previous
    public Object previous();

    // True if there is a previous element
    public boolean hasPrevious();

    // It is placed in the first element
    public void goFirst();

    // It is placed in the last element
    public void goLast();
}
```

- ExtendedIteratorImplementation
  - Código:

```java
public class ExtendedIteratorImplementation implements ExtendedIterator {
    private List<Sale> elements;
    private int currentIndex;

    public ExtendedIteratorImplementation(List<Sale> sales) {
        this.elements = sales;
        this.currentIndex = 0;
    }

    @Override
    public boolean hasNext() {
        return currentIndex < elements.size();
    }

    @Override
    public Object next() {
        if (!hasNext()) throw new NoSuchElementException();
        return elements.get(currentIndex++);
    }

    @Override
    public Object previous() {
        if (hasPrevious()) {
            currentIndex--;
            return elements.get(currentIndex);
        }
        return null;
    }

    @Override
    public boolean hasPrevious() {
        return currentIndex > 0;
    }

    @Override
    public void goFirst() {
        currentIndex = 0;
    }

    @Override
    public void goLast() {
        this.currentIndex = elements.size();
    }
}
```

- Domain
  - AdapterTest
    - Código:

```java
public class AdapterTest {

    public static void main(String[] args) {
        // Usamos createBLFacade() que es el método real que existe en tu BLFacadeFactory
        boolean isLocal = true;
        BLFacade blFacade = new BLFactory().getBusinessLogicFactory(isLocal);

        // Autenticar vendedor
        Seller s = blFacade.isLogged("Aitor Fernandez", "aurrera");

        // Crear la ventana de la tabla y hacerla visible
        SellerTable st = new SellerTable(s);
        st.setVisible(true);
    }
}
```

- Admin
  - Código:

```
package domain;

import java.io.Serializable;

@Entity
public class Admin implements Serializable {

    private static final long serialVersionUID =
    @XmlID
    @Id
    private String email;
    private String name;
    private String pass;

    public Admin() {}

    public Admin(String email, String name, Stri
        this.email = email;
        this.name = name;
        this.pass = pass;
    }

    public String getEmail() {
        return email;
    }

    public String getPass() {
        return pass;
    }

    public String getName() {
        return name;
    }
}
```

- Sale
  - Código:



- SellerAdapter
  - Código:

```java
package domain;

import javax.swing.table.AbstractTableModel;

public class SellerAdapter extends AbstractTableModel {

    private List<Sale> sales;

    // 1. Definimos exactamente los nombres de las columnas de la imagen objetivo
    private String[] columnNames = {"title", "description", "price"};

    public SellerAdapter(Seller seller) {
        if (seller != null && seller.getSales() != null) {
            this.sales = seller.getSales();
        }
    }

    @Override
    public int getRowCount() {
        return sales != null ? sales.size() : 0;
    }

    @Override
    public int getColumnCount() {
        return columnNames.length;
    }

    @Override
    public Object getValueAt(int rowIndex, int columnIndex) {
        Sale sale = sales.get(rowIndex);

        // 2. Mapeamos cada columna con el método getter correspondiente de Sale
        switch (columnIndex) {
            case 0: return sale.getTitle();       // Muestra el título ("futbol baloia", etc.)
            case 1: return sale.getDescription(); // Muestra la descripción
            case 2: return sale.getPrice();       // Muestra el precio (10.0, 20.0, etc.)
            default: return null;
        }
    }

    @Override
    public String getColumnName(int column) {
        return columnNames[column];
    }
}
```

- SellerTable
  - Código:

```
package domain;

import java.awt.BorderLayout;

public  class   SellerTable extends JFrame{
private Seller  seller;
private JTable  tabla;
public  SellerTable(Seller  seller){
super(seller.getName()+"'s  sales   ");
this.setBounds(100, 100,    700,    200);
this.seller =   seller;
SellerAdapter   adapt   =   new SellerAdapter(seller);
tabla   =   new JTable(adapt);
tabla.setPreferredScrollableViewportSize(new    Dimension(500,  70));
//Creamos   un  JscrollPane y   le  agregamos   la  JTable
JScrollPane scrollPane  =   new JScrollPane(tabla);
//Agregamos el  JScrollPane al  contenedor
getContentPane().add(scrollPane,    BorderLayout.CENTER);
}
}
```

- Gui
  - MainGui
    - Código:

```
public static void main(String[] args) {
    EventQueue.invokeLater(new Runnable() {
        public void run() {
            try {
                // 1. Inicializamos la lógica de negocio mediante la factoría
                boolean isLocal = true;
                BLFactory factory = new BLFactory();
                BLFacade appFacade = factory.getBusinessLogicFactory(isLocal);
                setBussinessLogic(appFacade);

                // 2. Lanzamos la ventana principal de la GUI
                MainGUI frame = new MainGUI("admin@gmail.com");
                frame.setVisible(true);
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    });
}
```

- TestIteratorMain
  - Código:

```
public class TestIteratorMain {

    public static void main(String[] args) {
        try {
            // 1. Obtenemos la lógica de negocio mediante la factoría
            boolean isLocal = true;
            BLFactory factory = new BLFactory();
            BLFacade blFacade = factory.getBusinessLogicFactory(isLocal);

            // (Omitimos initializeBD() para evitar duplicados en la base de datos)

            // 2. Obtenemos el iterador extendido pasando una cadena vacía ""
            ExtendedIterator it = (ExtendedIterator) blFacade.getSalesIterator("");
            Sale s;

            System.out.println("_____________________");
            System.out.println("FROM LAST TO FIRST");
            it.goLast(); // Ir al último elemento
            while (it.hasPrevious()) {
                s = (Sale) it.previous();
                System.out.println(s);
            }

            System.out.println();
            System.out.println("_____________________");
            System.out.println("FROM FIRST TO LAST");
            it.goFirst(); // Ir al primer elemento
            while (it.hasNext()) {
                s = (Sale) it.next();
                System.out.println(s);
            }

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
```

- ApplicationLauncher
  - Código:

```
public class ApplicationLauncher {
    private static final Logger LOGGER = Logger.getLogger(ApplicationLauncher.class.getName());

    public static void main(String[] args) {

        ConfigXML c = ConfigXML.getInstance();
        Locale.setDefault(new Locale(c.getLocale()));

        MainGUI a = new MainGUI("seller3@gmail.com");
        a.setVisible(true);



        try {


            UIManager.setLookAndFeel("javax.swing.plaf.metal.MetalLookAndFeel");
            BLFactory factory = new BLFactory();
            BLFacade blfacade = factory.getBusinessLogicFactory(c.isBusinessLogicLocal());


            MainGUI.setBussinessLogic(blfacade);

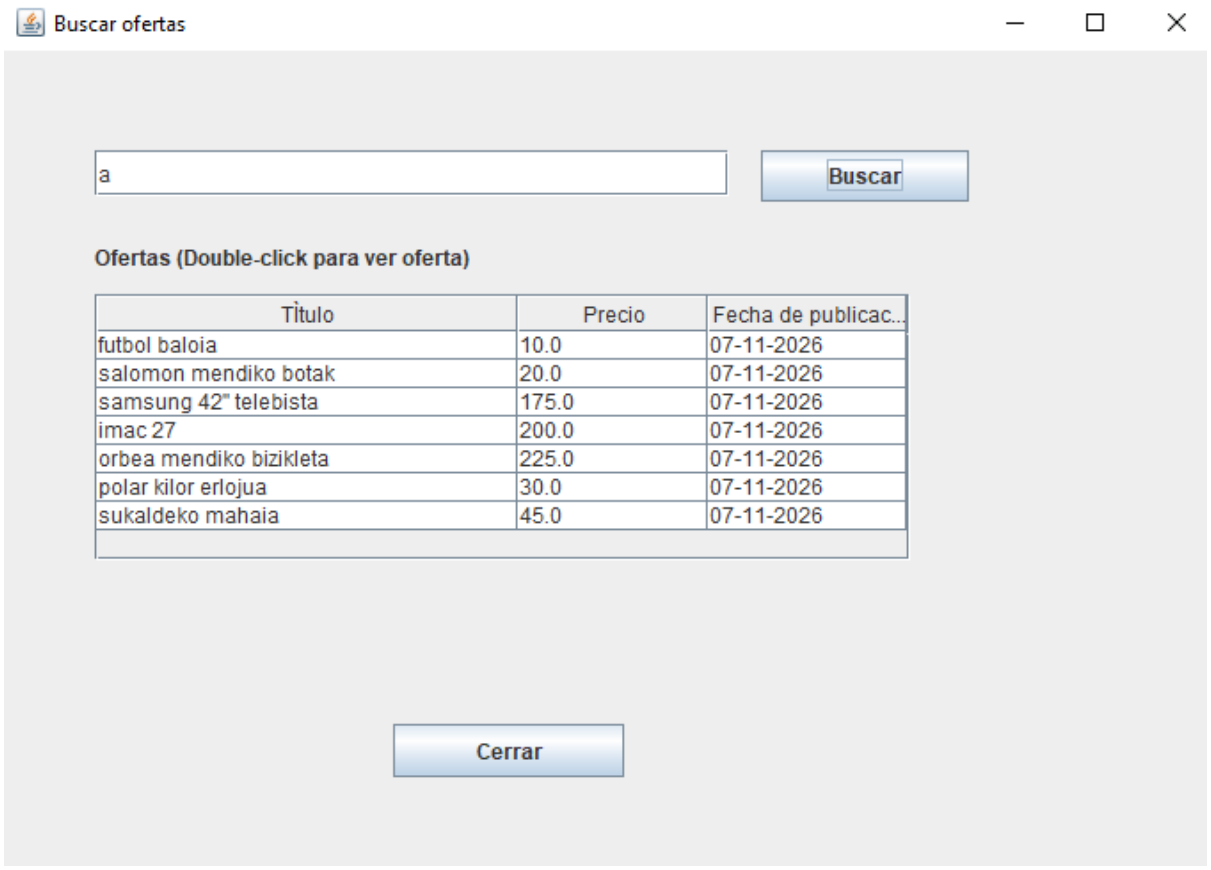
        }catch (Exception e) {
            a.jLabelSelectOption.setText("Error: "+e.toString());
            a.jLabelSelectOption.setForeground(Color.RED);

            LOGGER.log(Level.SEVERE, "Error in ApplicationLauncher", e);
        }


    }
```

C) Para los patrones Iterator y Adapter, una captura de imagen que muestre su ejecución.

ITERATOR RESULTADOS:



```
Read from config.xml:    businessLogicLocal=true        databaseLocal=true
File deleted
DataAccess opened => isDatabaseLocal: true
Db initialized
DataAccess created => isDatabaseLocal: true isDatabaseInitialized: true
DataAccess closed
Creating BLFacadeImplementation instance with DataAccess parameter
DataAccess opened => isDatabaseLocal: true
>> DataAccess: getProducts=> from=
DataAcess closed
_____________________
FROM    LAST    TO      FIRST
8;sukaldeko mahaia;45.0
7;polar kilor erlojua;30.0
6;orbea mendiko bizikleta;225.0
5;iphone 17;400.0
4;imac 27;200.0
3;samsung 42" telebista;175.0
2;salomon mendiko botak;20.0
1;futbol baloia;10.0


_____________________
FROM    FIRST   TO      LAST
1;futbol baloia;10.0
2;salomon mendiko botak;20.0
3;samsung 42" telebista;175.0
4;imac 27;200.0
5;iphone 17;400.0
6;orbea mendiko bizikleta;225.0
7;polar kilor erlojua;30.0
8;sukaldeko mahaia;45.0
```

ADAPTER RESULTADO:

